Chapter 6: Synchronization Tools

- Operating Systems | BS Computer Science
- Easy explanations, code, examples and classroom activities
- Based on the supplied Chapter 6 slides

Learning outcomes

- Explain race conditions and the critical-section problem
- Apply mutual exclusion, progress and bounded waiting
- Understand Peterson’s algorithm and hardware primitives
- Use mutexes, semaphores, monitors and condition variables

Why do processes need synchronization?

- Processes and threads may execute concurrently
- An interrupt can stop a process midway through an update
- Shared data may become inconsistent
- Synchronization coordinates access to shared resources

```
Two students editing the same shared file
```

Producer–consumer problem

- Producer puts items into a shared buffer
- Consumer removes items from the buffer
- Producer must wait when buffer is full
- Consumer must wait when buffer is empty

```
Bakery: baker produces bread; customer collects it
```

The shared counter

- counter = number of filled buffer slots
- Producer increments counter after adding an item
- Consumer decrements counter after removing an item
- Both update the same shared variable

```
Initially 5 items; one produced and one consumed → should remain 5
```

Producer algorithm

- Wait while buffer is full
- Place a new item in buffer[in]
- Advance in using modulo BUFFER_SIZE
- Increment counter

```
while (true) {
while (counter == BUFFER_SIZE) ;
  buffer[in] = next_produced;
 in = (in + 1) % BUFFER_SIZE;
    counter++;
}
```

Consumer algorithm

- Wait while buffer is empty
- Remove an item from buffer[out]
- Advance out using modulo BUFFER_SIZE
- Decrement counter

```
while (true) {
   while (counter == 0) ;
   next_consumed = buffer[out];
   out = (out + 1) % BUFFER_SIZE;
   counter--;
}
```

Race condition: meaning

- Multiple processes access and change shared data
- The final result depends on execution timing / interleaving
- counter++ is NOT necessarily one indivisible operation

```
Two students updating one bank balance
```

Race condition: how counter++ works

- Read counter into a register
- Add 1 to register
- Write register back to counter

```
register1 = counter;
register1 = register1 + 1;
counter = register1;
```

Race condition: step-by-step

- Initially counter = 5
- Producer reads 5 and calculates 6
- Consumer reads 5 and calculates 4
- Producer stores 6; consumer stores 4
- Final counter = 4, but correct answer = 5



Race condition in fork()

- Kernel stores the next available process ID
- Two processes may request a child PID at once
- Without mutual exclusion, both may read the same PID
- Synchronization prevents duplicate assignment

```
Two parents calling fork() at the same moment
```

What is a critical section?

- Part of a process that accesses shared data
- Examples: changing a counter, writing a file, updating a table
- Only one process may execute a protected critical section at a time

```
Shared printer management
```

Four sections of a process

- ENTRY: request permission
- CRITICAL: use shared resource
- EXIT: release permission
- REMAINDER: perform unrelated work

```
while (true) {
  // entry section
 // critical section
  // exit section
// remainder section
        }
```

Requirement 1: Mutual exclusion

- If P0 is inside its critical section, P1 cannot enter
- Prevents overlapping changes to shared data

```
Only one student holds the laboratory key
```

Requirement 2: Progress

- When the critical section is free, waiting processes should be considered
- Selection of the next entrant cannot be postponed indefinitely

```
If the printer is idle, let a waiting student use it
```

Requirement 3: Bounded waiting

- A requesting process must not be bypassed forever
- There must be a limit on how many times others enter first

```
No student should remain permanently at the end of a queue
```

Kernel: preemptive vs non-preemptive

- Preemptive kernel: a running kernel-mode task may be preempted
- Non-preemptive kernel: continues until it exits, blocks or yields
- Kernel design affects synchronization requirements

```
A teacher interrupted midway versus finishing an instruction
```

Peterson’s solution: idea

- Classic software solution for TWO processes
- Uses flag[2] to show interest
- Uses turn to break a tie
- Assumes suitable atomic load/store and memory ordering

```
P0 and P1 both want to enter a protected area
```

Peterson’s solution: variables

- flag[i] = true means Pi wants to enter
- turn = j gives the other process priority if both want entry
- Pi waits only while Pj wants entry AND turn == j

```
boolean flag[2] = {false, false};
int turn;
```

Peterson’s solution: algorithm

- Set own flag true
- Give the other process the turn
- Wait while the other process is interested and has priority
- After critical section, clear own flag

```
while (true) {
 flag[i] = true;
    turn = j;
while (flag[j] && turn == j) ;
      // critical section
  flag[i] = false;
      // remainder section
}
```

Why Peterson’s solution works in its model

- Mutual exclusion: both cannot enter simultaneously
- Progress: if other process is uninterested, enter
- Bounded waiting: turn resolves competition
- IMPORTANT: not guaranteed on modern hardware as written

```
Discuss why compiler/CPU instruction ordering matters
```

Instruction reordering problem

- Processors and compilers may reorder independent operations
- A single-threaded result may stay the same
- Other threads may observe an unexpected order
- This can invalidate naive synchronization code

```
Expected x=100; a reordering can expose x=0
```

Memory ordering example

- Thread 2 writes x=100 and then flag=true
- Thread 1 waits for flag and prints x
- Without correct ordering, seeing flag does not always ensure seeing x

```
// Thread 1
while (!flag) ;
print(x);

// Thread 2
x = 100;
flag = true;
```

Hardware synchronization support

- Disabling interrupts can help in limited uniprocessor kernel contexts
- Not scalable as a general multiprocessor solution
- Three main tools: memory barriers, special atomic instructions, atomic variables

Memory models and barriers

- Strong ordering: writes become visible in a tightly ordered way
- Weak ordering: visibility and order may be delayed
- Memory barriers enforce needed ordering/visibility constraints
- Correct use depends on the architecture and memory model

```
A note written before “ready” should be seen before the ready flag
```

Test-and-set instruction

- Atomically reads the old lock value and sets it to true
- Returns the OLD value
- A false old value means the lock was free

```
boolean test_and_set(boolean *target) {
        boolean old = *target;
        *target = true;
        return old;
        }
```

Lock using test-and-set

- Shared lock initially false
- Repeatedly test-and-set until old value is false
- Run critical section; set lock=false on exit
- The repeated checking is busy waiting

```
        while (true) {
while (test_and_set(&lock)) ;
      // critical section
         lock = false;
      // remainder section
              }
```

Compare-and-swap (CAS)

- Atomically compare current value with expected value
- Update only if values match
- Return the original value
- Often used to build locks and atomic variables

```
int compare_and_swap(int *v, int expected, int next) {
                        int old = *v;
                if (*v == expected) *v = next;
                        return old;
                            }
```

Lock using compare-and-swap

- lock = 0 means unlocked; lock = 1 means locked
- CAS attempts to change 0 → 1
- If original value is not 0, keep waiting
- Release by setting lock back to 0

```
while (true) {
while (compare_and_swap(&lock, 0, 1) != 0) ;
// critical section
lock = 0;
}
```

Bounded-waiting CAS solution

- Basic spinlocks do not guarantee bounded waiting
- The slide algorithm adds a waiting[] array
- After exiting, a process hands over to a waiting successor
- Successor search wraps around with (i + 1) % n

```
A circular queue gives waiting processes a turn
```

Atomic variables

- Provide indivisible operations on basic types
- Examples: atomic integers and booleans
- Hardware CAS can implement atomic increment
- Useful for shared counters

```
                                    do {
                                  temp = *v;
        } while (temp != compare_and_swap(v, temp, temp + 1));
```

Mutex lock: simple idea

- Mutex = mutual exclusion
- acquire() before the critical section
- release() after the critical section
- Lock operations must be atomic

```
One key for one shared laboratory room
```

Mutex example

- P1 acquires the lock and updates counter
- P2 cannot enter until P1 releases the lock
- This prevents lost updates

```
while (true) {
 acquire(lock);
   counter++;
 release(lock);
// remainder section
       }
```

Spinlocks and busy waiting

- A spinlock repeatedly checks for availability
- Waiting process may use CPU time without useful work
- Can be useful for very short waits
- Blocking can be better for long waits

```
Student repeatedly asks “Is the printer free yet?”
```

Semaphore: definition

- An integer synchronization variable
- Accessed using atomic wait() and signal()
- wait() requests a permit; signal() returns one
- Can coordinate access AND order of execution

```
A lab with 3 available computers
```

Binary vs counting semaphores

- Binary semaphore: 0 or 1
- Counting semaphore: number of available permits
- A binary semaphore can act like a lock
- Semaphore signaling does not necessarily require ownership

```
One printer versus three printers
```

Semaphore wait() and signal()

- wait(S): if no permit, wait; otherwise decrement
- signal(S): increment and potentially wake a waiter
- The operations must be atomic
- Slide first shows a busy-wait implementation

```
wait(S) {
while (S <= 0) ;
S--;
}
signal(S) { S++; }
```

Counting semaphore: walk-through

- Start S=3 (three identical printers)
- P1 wait → 2; P2 wait → 1; P3 wait → 0
- P4 wait → blocked
- P1 signal → a permit is available for a waiter

```
Have four students act as processes
```

Semaphore for process ordering

- Suppose S1 must finish before S2 starts
- Initialize synch = 0
- P1 performs S1 then signal(synch)
- P2 waits on synch before S2

```
        // P1                    // P2
S1;                         wait(synch);
     signal(synch);              S2;
```

Semaphore implementation: no busy waiting

- Each semaphore has a waiting queue
- block() suspends the requesting process
- wakeup() moves a waiting process to ready state
- Avoids continuous CPU polling

```
P4 sleeps in the queue until a printer is released
```

Blocking semaphore pseudocode

- A negative value can represent waiting processes in this model
- wait() decrements and blocks if value < 0
- signal() increments and wakes a waiter if value <= 0

```
wait(S) {
S->value--;
if (S->value < 0) { enqueue(S); block(); }
}
signal(S) {
S->value++;
if (S->value <= 0) { dequeue(S); wakeup(); }
}
```

Common semaphore mistakes

- Calling signal() before wait() incorrectly
- Calling wait() twice without a matching release
- Forgetting wait() or signal()
- These mistakes can cause incorrect access or indefinite waiting

```
Ask students: what happens if a process never signals?
```

Monitor: high-level synchronization

- Encapsulates shared variables and procedures
- Only one process/thread active inside a monitor at a time
- Provides a more structured synchronization approach
- Uses condition variables for waiting and notification

A bank service counter managed by one office

Monitor structure

- Shared state is private to monitor procedures
- Procedures operate on protected shared data
- Initialization establishes starting state

```
monitor Example {
// shared variables
function P1(...) { ... }
function P2(...) { ... }
 initialization { ... }
}
```

Condition variables

- x.wait(): suspend a process until signaled
- x.signal(): wake one waiting process, if any
- If nobody is waiting, signal has no stored effect
- Used inside monitors to coordinate conditions

```
Consumer waits until the buffer is not empty
```

Signal-and-wait vs signal-and-continue

- Signal-and-wait: signaler P waits while Q runs
- Signal-and-continue: P continues, Q waits for entry
- Both must preserve monitor mutual exclusion
- Exact semantics depend on implementation

```
Who gets the service counter after the signal?
```

Monitor implementation with semaphores

- The slides use mutex initialized to 1
- next initialized to 0; next_count tracks urgent waiters
- Each monitor procedure waits on mutex before entry
- Exit signals next or mutex depending on waiting state

```
wait(mutex);
   // body of monitor procedure
if (next_count > 0) signal(next);
     else signal(mutex);
```

Condition-variable implementation

- Each condition x has x_sem initially 0
- x_count counts processes waiting on x
- x.wait() releases monitor access and blocks
- x.signal() wakes a waiter with appropriate handoff

CLASS EXAMPLE | Important: handoff rules differ between monitor implementations

Which waiting process resumes?

- Several processes may wait on one condition
- FCFS is not always suitable
- Slides describe conditional wait x.wait(c)
- Lower numeric c represents higher priority

```
Emergency jobs may be given priority
```

Liveness: what should we remember?

- Processes should keep making progress
- Indefinite waiting is undesirable
- Poor synchronization may cause deadlock or starvation
- The supplied slides list liveness as a topic but do not detail it

```
Ask: what if everyone waits and nobody signals?
```

Choosing a synchronization tool

- Atomic variables: small shared updates
- Mutex: protect a single critical section
- Counting semaphore: multiple resource permits
- Monitor + condition variable: structured coordination
- Choose based on contention and wait duration

```
Short waits may spin; long waits often benefit from blocking
```

Class activity 1: race condition

- Start counter = 5
- Student A acts as producer; B as consumer
- Both read 5 before writing
- A writes 6; B writes 4
- Explain why the expected final value is 5

Class activity 2: mutex and semaphore

- Three students represent three printers
- Four students request access using wait()
- Identify which process blocks
- One printer is released using signal()
- Explain how mutex differs from a counting semaphore

Quick quiz

- What causes a race condition?
- State all three critical-section requirements
- What do flag and turn mean in Peterson’s solution?
- What is the difference between test-and-set and CAS?
- Compare mutex, semaphore, monitor and condition variable

Summary: the big picture

- Problem: concurrent changes can cause race conditions
- Goal: safe critical sections and coordinated processes
- Low-level tools: memory barriers, atomic operations
- Higher-level tools: mutexes, semaphores, monitors
- Remember: correctness, progress, and fair waiting

Thank you — Questions?

- Chapter 6: Synchronization Tools
- Next practice: trace code and solve synchronization scenarios